Strengthening Test Security on Windows: Draft Technical Recommendations

Date: March 2026
Author: Microsoft
Version: 0.2

---

1. Introduction

Purpose

This document provides technical recommendations for strengthening the security of testing applications/solutions on Windows. Recommendations are organized into two tiers:

- **Baseline (user-mode):** All mitigations use existing, documented Windows APIs and policies. No kernel driver is required. This tier can be implemented with the Bluebook app and the Bluebook Security Service.
- **Enhanced (kernel-mode driver):** A signed kernel-mode driver component provides substantially stronger protections — particularly for handle protection, real-time injection detection, and tamper resistance. Section 4d discusses these benefits in detail.

Both tiers use existing Windows platform capabilities available today — no unreleased features are required.

Scope

- **Target platforms:** Windows 10 21H2+ and Windows 11 (all editions)
- **Architecture assumption:** Bluebook is a user-mode Win32 or UWP application running on BYOD student laptops where students may have local administrator privileges and no MDM enrollment

Architecture Overview and Flow

The recommended ideal architecture has four components. All client-side components can be built, signed, and shipped by College Board as part of the Bluebook installation package. A key challenge is that some of the mitigations will require admin privileges. The recommended architecture describes the creation of a distinct service where there is a dependency on admin privilege while the Bluebook app itself can still run purely as standard user. Alternatively running the app as admin would obviate the need for the service.

Bluebook app — The test-taking application itself, running as the logged-in student (standard user privileges). Implements all self-hardening mitigations that require no elevation: display affinity, process mitigation policies, clipboard isolation, input injection detection, VM checks, and debugger detection. These are the highest-priority (P0) items.

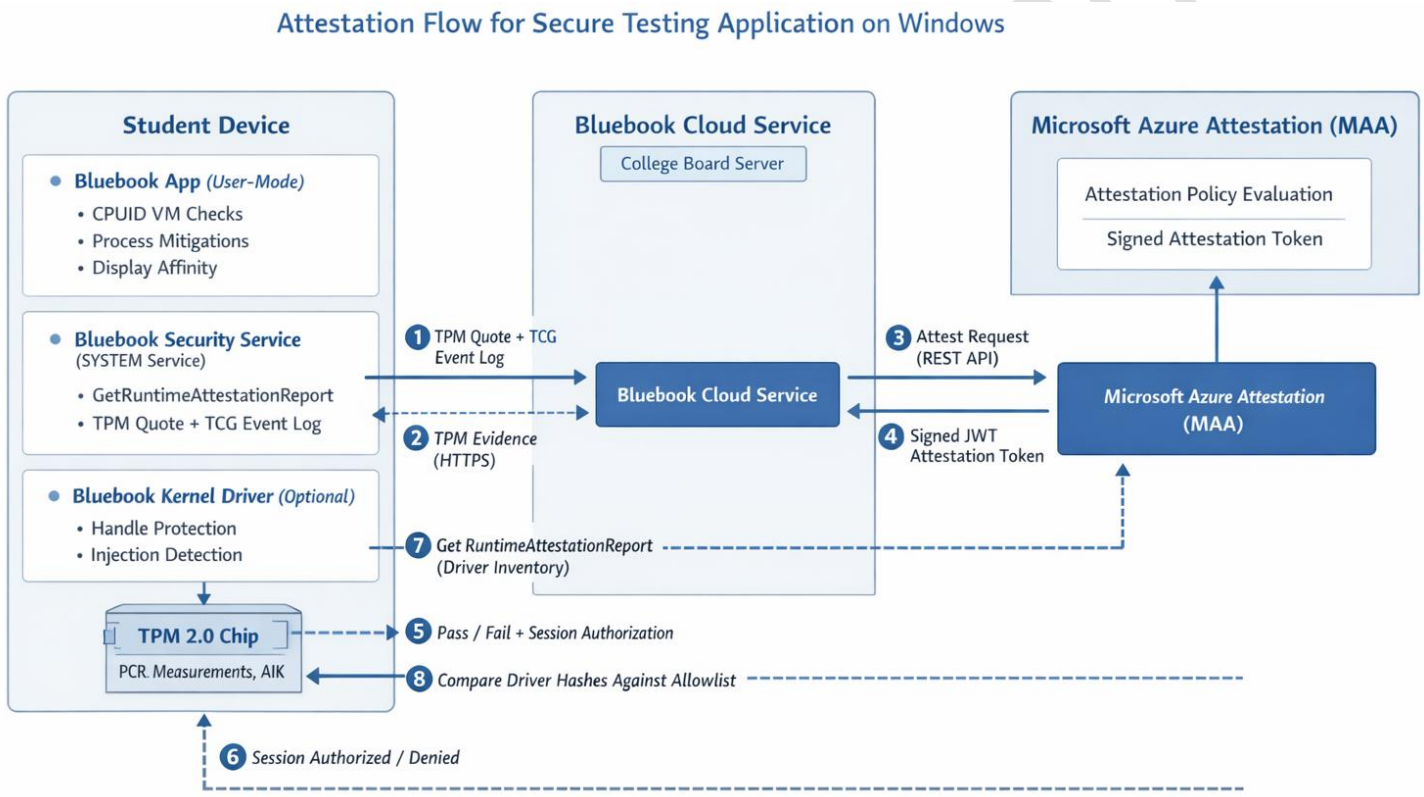
Bluebook Security Service — A Windows service installed alongside the Bluebook app, running as SYSTEM. Activated on demand when a test session starts; idle otherwise. Handles all mitigations

Microsoft Confidential

requiring elevated privileges: WFP network rules, MAA attestation evidence collection, App Control for Business (fka WDAC) policy deployment, USB/audio/NPU policy, and ETW-based process monitoring. Communicates with the Bluebook app via a local named pipe or localhost socket. Does not require MDM or domain join.

Bluebook Kernel Driver *(Enhanced tier, optional)* — A Microsoft-signed kernel-mode driver that provides kernel-level handle protection, real-time process/DLL load monitoring, and tamper resistance for the security service. See Section 4d for a detailed discussion of what this component enables. This is a significant engineering investment but closes critical gaps that user-mode mitigations cannot address.

Bluebook Cloud Service — College Board's existing server infrastructure, extended with attestation validation capabilities. Receives TPM evidence from the security service, calls Azure Attestation for policy evaluation, maintains the driver hash allowlist, and makes pass/fail decisions for test session authorization.



Defense-in-Depth

No single mitigation is sufficient against a determined attacker running code with local admin privileges. Each layer raises the cost, complexity, and skill required to cheat. The goal is to make Windows-based cheating economically unattractive relative to the value of the test, and to detect compromise when prevention fails.

Classified as Microsoft Confidential

2. Detecting Virtualized Environments

Problem

A virtual machine allows the host operating system to observe all guest memory, display output, keyboard input, and network traffic. Any in-guest security measure — process protection, screen capture prevention, code integrity — can be trivially bypassed by the host. VM detection is therefore a prerequisite for all other mitigations.

Detection Techniques

The following techniques are ranked by reliability and difficulty of spoofing:

Technique	Notes
Timing Analysis	Many options. Simple tight loop timing comparison – 2 threads both add but one also has CPUID instr. Later compare accumulation from each thread and will see any virtualization discrepancy. Can use perf monitoring counters or other timing side channel options <Discussion Topic> Standard user. Windows 10 and 11.
TPM AIK certificate verification via MAA	VMs lack valid hardware AIK certificates. However, a custom hypervisor can relay TPM commands to the host's physical TPM, returning valid quotes and AIK credentials. Relay requires hypervisor development skills but is not cryptographically difficult. Strongest available signal, but not airtight against a determined attacker. *Requires admin/SYSTEM (TPM APIs). Windows 10 and 11.*

Primary Recommendation: MAA + TPM AIK Verification

Microsoft Azure Attestation (MAA) can verify that a device has a hardware TPM with a valid Attestation Identity Key (AIK) certificate. Virtual machines — including Hyper-V, VMware, and VirtualBox guests — lack valid hardware AIK certificates issued by TPM manufacturers.

Implementation flow:

1. Before test session starts, the security service collects a TPM quote and TCG event log
2. Service sends this evidence to the CB attestation server
3. Bluebook cloud service forwards to MAA for evaluation against a configured attestation policy
4. MAA returns a signed attestation token indicating whether the device has a valid hardware TPM
5. If no valid AIK certificate is present → block test session (device is likely a VM)

This is the strongest available VM detection technique against standard virtualization. However, a sophisticated attacker with a custom hypervisor can relay TPM operations to the host's physical hardware, bypassing this check without cryptographic forgery — the attacker does not need to fake an AIK, they simply forward commands to a real TPM and return the genuine responses. Layering multiple independent detection signals (CPUID, SMBIOS, timing, runtime attestation) increases the cost of maintaining a consistent relay across all channels.

This capability is available today for any device with a TPM 2.0 chip. See the Azure Attestation documentation for details.

Microsoft Confidential

TPM relay attack mitigation: To increase relay difficulty, the attestation protocol should bind TPM quotes to the TLS session with the Bluebook cloud service (include a hash of the TLS session key in the attestation nonce) and perform multiple rapid challenge-response rounds during the test session to accumulate detectable latency in a relay scenario. These measures raise the bar but do not eliminate the relay threat — see Section 10 for a full discussion of residual risk.

---

3. Attesting Boot and OS Integrity

Problem

A device may be running on bare metal but with security features disabled: Secure Boot turned off, test signing enabled, HVCI disabled, or vulnerable kernel drivers loaded. Attestation verifies the device's security configuration before allowing a test session.

3a. Measured Boot via Microsoft Azure Attestation (MAA)

How it works: During every boot, firmware and the Windows bootloader measure each component in the boot chain — UEFI firmware, bootloader, OS kernel, early-launch anti-malware drivers, WDAC policy state — and extend these measurements into TPM Platform Configuration Registers (PCRs). These measurements, combined with a TPM-signed quote, form unforgeable evidence of the device's boot state.

Attestation flow:

1. Bluebook Security Service calls TPM APIs to collect the TPM quote (signed PCR values) and the TCG event log (detailed record of what was measured)
2. Service sends evidence to the CB attestation server
3. Bluebook cloud service calls MAA's `attest` endpoint with the evidence
4. MAA evaluates the evidence against the CB-defined attestation policy
5. MAA returns a signed JWT attestation token containing evaluated claims
6. Bluebook cloud service inspects claims and makes a pass/fail decision

Claims to validate in attestation policy:

Claim	Expected Value	Why
Secure Boot	Enabled	Mitigation for boot-time rootkits
Code Integrity policy	Active and enforced	AppControl is in enforcement mode
HVCI	Enabled	Hypervisor protects kernel code integrity
Test signing	Disabled	Prevents loading of unsigned test-signed drivers
Kernel debugging	Disabled	Prevents debugger-based memory inspection
Driver Signature Enforcement (DSE)	Active	Only signed drivers can load
Boot debugging	Disabled	Prevents boot-time debugger attachment

MAA policy is configurable. College Board defines the minimum acceptable boot state.

Microsoft Confidential

Availability: MAA is a generally available Azure service. The client-side TPM APIs are built into Windows 10 and 11. No special licensing is required.

Windows 10 and 11.

3b. Runtime Driver Attestation — GetRuntimeAttestationReport

When HVCI (Hypervisor-Enforced Code Integrity) is enabled, Windows provides the `GetRuntimeAttestationReport` API that returns a hypervisor-signed report of **every kernel module that has ever been loaded** during the current boot session.

What the report contains: For each driver, a `_DRIVER_INFO_ENTRY` structure provides:

Field	Description
Driver name	e.g., `ntoskrnl.exe`, `ndis.sys`
Image SHA-256 hash	Hash of the driver binary as loaded
Leaf certificate SHA-1	Signing certificate thumbprint
OEM name	Publisher name from certificate
Load count	Number of times loaded
Unload count	Number of times unloaded
Status flags	Current state of the module

Key security property: Even drivers that have been loaded and then unloaded appear in the report. This defeats the "load vulnerable driver → exploit → unload → clean up" attack pattern commonly used by cheat tools.

Implementation:

1. Bluebook Security Service calls `GetRuntimeAttestationReport` during or after the test session
2. Report is sent to the Bluebook cloud service
3. Bluebook cloud service compares driver hashes against an allowlist of known-good drivers
4. Any unknown or known-bad driver hash triggers investigation or score invalidation

Requirement: HVCI must be enabled on the device. This can be verified via MAA boot attestation (Section 3a) before relying on this API.

Requires admin/SYSTEM. Windows 11 only (requires HVCI enabled).

3c. App Control Code Integrity Policies for Test Sessions

App Control can block known-bad user-mode and kernel-mode code from executing. For Bluebook test sessions, an App Control policy can restrict the software environment during the test.

Test session workflow:

1. Before test: security service deploys an App Control policy
2. [Optional] Require device reboot to ensure all processes, including any currently running, are subject to policy enforcement
3. Student takes test with restricted software environment

Microsoft Confidential

4. After test: security service removes the policy
5. Device reboots to restore normal operation

Policy capabilities:

- Block known cheat tool executables and DLLs by hash, file properties from the file's RSRC header file, file path, etc...
- Block unsigned or untrusted binaries from executing, e.g. prevent known script hosts from launching.
- Allow only Microsoft-signed and CB-signed code (strictest mode)
- Audit mode available for testing before enforcement
- Set custom policy to control what is allowed to load into a binary or to prevent loading specific things into the process. Similar to Code Integrity Guard (CIG) described in the next section but policy defined by the College Board. See Use an App Control for Business policy to control specific plug-ins, add-ins, and modules.

College Board maintains and updates the blocklist as new cheat tools emerge. Hash-based blocking is precise but requires ongoing intelligence. Publisher certificate-based rules (e.g., "block all code signed by known cheat tool publisher") provide broader coverage. App Control policy is a block by default model, so College Board's policy must include rules to allow the code that should be trusted. All other code is implicitly blocked.

An "Allow *" rule can be used along with explicit deny rules to block only known bad code while allowing all other code. Policy using allow rules only is the most restrictive as only the code you explicitly authorize runs. Depending on the assurances you wish to make, you may need to trade-off between the policy strength and usability.

Known limitations:

- Enforcement for processes already running requires a reboot (or process restart) to activate. Recommended for test scenario based on threats.
- Full policy removal requires reboot. Recommend replacing policy with an "Allow *" policy after test completion then delete to allow removal on reboot.
- Students with local admin can remove the policy — combine with attestation to detect this.

Requires admin/SYSTEM for policy deployment. Windows 10 1903+ and Windows 11.

---

4. Preventing Code Injection

Problem

The three active attack families targeting Bluebook all rely on code injection:

- **Co-skip:** Injects into `dwm.exe` (Desktop Window Manager) to capture screen content, sends screenshots to a command-and-control server for AI-powered answer inference
- **Genie Study:** Uses a reflective loader injected via `ctfmon.exe` (the Text Services Framework process) to execute cheat code without touching disk

Microsoft Confidential

- **Furina:** Injects DLLs directly into the Bluebook process to read test content from memory

4a. Hardening the Bluebook Process

The `SetProcessMitigationPolicy` API allows an application to opt into a suite of process-level security mitigations. These must be applied **at process startup, before any external code has a chance to load.**

Recommended mitigations:

Mitigation	API Constant	Effect
Arbitrary Code Guard (ACG)	`ProcessDynamicCodePolicy`	Prevents modification of executable code pages. Blocks shellcode injection techniques that write to memory and then execute it.
Code Integrity Guard (CIG)	`ProcessSignaturePolicy`	Only Microsoft-signed (or WHQL-signed) DLLs can be loaded into the process. Directly defeats the Furina attack — unsigned cheat DLLs cannot load.
Control Flow Guard (CFG)	`ProcessControlFlowGuardPolicy`	Restricts indirect call targets to valid function entry points. Prevents Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP) chains used by sophisticated injectors.
Binary Signature Policy	`ProcessSignaturePolicy`	Restrict loaded images to Microsoft-signed binaries. Can be configured to also allow CB-signed code.
Dynamic Code Policy	`ProcessDynamicCodePolicy`	Prevents `VirtualAlloc` with `PAGE_EXECUTE` permissions in the process, blocking runtime code generation used by reflective loaders.

Implementation (C/C++):

```
// Enable Code Integrity Guard — only MS-signed DLLs
PROCESS_MITIGATION_BINARY_SIGNATURE_POLICY sigPolicy = {0};
sigPolicy.MicrosoftSignedOnly = 1;
SetProcessMitigationPolicy(ProcessSignaturePolicy,
    &sigPolicy, sizeof(sigPolicy));

// Enable Arbitrary Code Guard — no dynamic code
PROCESS_MITIGATION_DYNAMIC_CODE_POLICY dynPolicy = {0};
dynPolicy.ProhibitDynamicCode = 1;
SetProcessMitigationPolicy(ProcessDynamicCodePolicy,
    &dynPolicy, sizeof(dynPolicy));
```

Microsoft Confidential

Critical: All mitigations must be applied at process initialization, before `DllMain` of non-system DLLs executes. Once a cheat DLL has loaded, it is too late.

Standard user — process applies these to itself. Windows 10 1709+ and Windows 11.

4b. Protecting System Processes (DWM, CTFMon)

The Bluebook application cannot directly harden other system processes (dwm.exe, ctfmon.exe) — these are owned by the operating system. However, several approaches can mitigate injection into these processes:

Baseline:

- **Exploit Protection (per-process mitigations via IFEO):** The Bluebook Security Service can configure Microsoft Defender Exploit Protection mitigations for dwm.exe and ctfmon.exe by writing to the Image File Execution Options (IFEO) registry keys. These mitigations are enforced by the kernel at process creation time and **directly prevent code injection** — not just detect it. This is the strongest baseline mitigation for system processes and does not require a kernel driver. See Section 4b-i below for details.
- **AppControl allowlist policy (Section 3c):** A strict App Control policy allowing only Microsoft-signed and CB-signed user-mode code prevents unsigned cheat payloads from loading into any process — including dwm.exe and ctfmon.exe. This is system-wide, not per-process, but directly blocks DLL and executables loading. App Control enforcement is done as part of process creation and doesn't prevent direct memory modification but that can be prevented using other mitigations listed in this section. With HVCI enabled, AppControl also prevents shellcode/reflective injection by enforcing W^X (write XOR execute) for kernel mode pages at the hypervisor level.
- **Attestation (Sections 3a–3b):** Detect post-facto whether suspicious code was loaded during the test session. If the runtime driver attestation report shows an unknown driver was loaded, or boot attestation reveals security features were disabled, the test result can be flagged.
- **ETW monitoring:** The Bluebook Security Service can subscribe to ETW (Event Tracing for Windows) events for process creation and DLL loading. Suspicious activity — such as an unknown process loading DLLs into dwm.exe — can be logged and reported to the Bluebook cloud service.

Enhanced (with kernel driver — see Section 4d):

- **ObRegisterCallbacks:** Strip `VM_READ` and `VM_WRITE` access rights from handles opened to system processes. Prevents cheat tools from injecting code into dwm.exe or ctfmon.exe.
- **PsSetCreateProcessNotifyRoutineEx:** Monitor and optionally block process creation of known-bad executables in real time — before the cheat tool has a chance to run.
- **PsSetLoadImageNotifyRoutine:** Detect DLL loads into system processes as they happen. If an unknown DLL is loaded into dwm.exe during a test session, the driver can alert the security service immediately.

With Exploit Protection mitigations, the baseline approach now includes prevention for system processes — not just detection. The kernel driver adds real-time handle protection and process creation blocking as additional layers.

Microsoft Confidential

4b-i. Exploit Protection Mitigations for System Processes

Microsoft Defender Exploit Protection allows per-process security mitigations to be configured via the IFEO registry key. The Bluebook Security Service (running as SYSTEM) writes these settings before the test session; they take effect when the target process next starts (requires a reboot for dwm.exe).

Mitigations for dwm.exe:

Mitigation	PowerShell Parameter	Effect	Compatibility Risk
Code Integrity Guard (CIG)	`BlockNonMicrosoftSigned`	Only Microsoft-signed and WHQL-signed DLLs can load. Directly blocks Co-skip's unsigned injection payload.	DWM loads Microsoft system DLLs and WHQL-signed GPU drivers. Some OEM display helper DLLs could be blocked.
Arbitrary Code Guard (ACG)	`DisallowDynamicCode`	Prevents dynamic executable code generation. Blocks shellcode and reflective DLL injection.	GPU drivers may use JIT-compiled shaders. Requires per-GPU-vendor testing.
Disable Extension Points	`DisableExtensionPoints`	Blocks AppInit_DLLs, Winsock LSPs, and legacy hooking mechanisms used for injection.	DWM does not depend on these extension points.
Block Low Integrity Images	`BlockLowLabel`	Prevents loading DLLs with low integrity labels.	DWM runs as SYSTEM and should not load low-integrity images.
Block Remote Images	`BlockRemoteImageLoads`	Prevents loading DLLs from network/UNC paths.	DWM should not load DLLs from network paths.
Export Address Filtering (EAF)	`EnableExportAddressFilter`	Guards export address table reads in kernel32/ntdll, making it harder for injected shellcode to resolve API addresses.	Rarely causes functional issues.

Mitigations for ctfmon.exe:

Mitigation	PowerShell Parameter	Compatibility Risk
Disable Extension Points	`DisableExtensionPoints`	Needs testing with legacy IME loading
Block Low Integrity Images	`BlockLowLabel`	Low
Block Remote Images	`BlockRemoteImageLoads`	Low
Export Address Filtering (EAF)	`EnableExportAddressFilter`	Low

CIG is NOT recommended for ctfmon.exe in the general case. ctfmon.exe is designed to load third-party Text Service Framework (TSF) components and input method editors (IMEs). Enabling CIG would block non-Microsoft input methods, breaking multilingual input for international students. If CB can confirm that no third-party IMEs are needed during the test, CIG can be enabled for ctfmon.exe as well.

Implementation:

Microsoft Confidential

The Bluebook Security Service writes the IFEO registry key directly — no PowerShell dependency required:

```
// Registry path for dwm.exe mitigations
// HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\
//   Image File Execution Options\dwm.exe\MitigationOptions
//
// MitigationOptions is a REG_BINARY bitmask of enabled mitigations.
// MitigationAuditOptions uses the same format for audit-only mode.
```

Alternatively, via PowerShell:

```
# Deploy mitigations for dwm.exe
Set-ProcessMitigation -Name dwm.exe -Enable `
    BlockNonMicrosoftSigned, `
    DisallowDynamicCode, `
    DisableExtensionPoints, `
    BlockLowLabel, `
    BlockRemoteImageLoads, `
    EnableExportAddressFilter

# Deploy mitigations for ctfmon.exe (conservative set)
Set-ProcessMitigation -Name ctfmon.exe -Enable `
    DisableExtensionPoints, `
    BlockLowLabel, `
    BlockRemoteImageLoads, `
    EnableExportAddressFilter
```

Test session workflow:

1. Before test: Security service writes IFEO mitigation settings for dwm.exe and ctfmon.exe
2. Reboot to apply (dwm.exe mitigations take effect when the process starts; this reboot can be combined with the AppControl policy activation reboot)
3. Student takes test with hardened system processes
4. After test: Security service removes IFEO mitigation settings
5. Reboot to restore normal operation

Audit mode: Before enforcement, College Board should deploy mitigations in **audit mode** (`MitigationAuditOptions` registry key) to identify compatibility issues. Audit events are logged to the Windows Event Log under `Microsoft-Windows-Security-Mitigations` and can be collected via ETW. This allows testing across diverse hardware (GPU vendors, input methods) without risking breakage.

Limitations:

- Mitigations apply at process creation time, not to already-running instances. A reboot is required for dwm.exe.
- A local administrator can remove the IFEO registry settings. Combine with attestation to detect tampering, or protect the registry keys with the kernel driver (`CmRegisterCallbackEx`) in the Enhanced tier.
- CIG allows WHQL-signed DLLs, not just Microsoft-signed. A cheat tool distributed as a WHQL-signed driver DLL (unlikely but theoretically possible) would not be blocked by CIG alone.

Microsoft Confidential

Requires admin/SYSTEM to write IFEO registry keys. Windows 10 1709+ and Windows 11.

4c. Handle and Memory Access Prevention

Cheat tools commonly open handles to the Bluebook process using `OpenProcess` with `PROCESS_VM_READ` or `PROCESS_VM_WRITE` access to read test content from memory.

Baseline mitigations:

- `SetProcessMitigationPolicy` with the recommended mitigations (Section 4a) significantly reduces the attack surface
- As a detection measure: Bluebook can periodically enumerate handles open to its own process via `NtQuerySystemInformation(SystemHandleInformation)` and flag unexpected handles. *Standard user (self-query). Windows 10 and 11.*

Enhanced (with kernel driver):

- `ObRegisterCallbacks` allows the kernel driver to intercept every `OpenProcess` and `OpenThread` call targeting the Bluebook process. The driver can strip `PROCESS_VM_READ`, `PROCESS_VM_WRITE`, and `PROCESS_VM_OPERATION` access rights from handles requested by non-trusted processes. This is the **strongest available protection** against memory reading and code injection short of Protected Process Light (PPL) — it operates at the kernel level and cannot be bypassed by user-mode code. *Requires kernel-mode driver. Windows 10 and 11.*

4d. The Case for a Kernel-Mode Driver

The baseline (user-mode) mitigations in this document provide meaningful security improvements. However, the fundamental challenge of the Bluebook threat model — the device owner has local admin privileges — means that user-mode protections have inherent limits. A user-mode service running as SYSTEM can be stopped, modified, or bypassed by an admin-level attacker. A signed kernel driver significantly raises this bar.

What the kernel driver enables that user-mode cannot:

Capability	Without Driver (Baseline)	With Driver (Enhanced)
Handle protection for Bluebook process	Detection only (`NtQuerySystemInformation` polling)	**Prevention** — `ObRegisterCallbacks` strips dangerous access rights from every handle at creation time
DLL injection prevention for system processes (dwm.exe, ctfmon.exe)	**Prevention** — Exploit Protection (CIG, ACG, Disable Extension Points) via IFEO blocks unsigned DLLs, shellcode, and legacy injection hooks. Requires reboot.	**Prevention + real-time detection** — `PsSetLoadImageNotifyRoutine` fires on every image load into any process, complementing Exploit Protection with immediate alerting
Process creation blocking	AppControl (requires reboot for kernel enforcement)	**Real-time** — `PsSetCreateProcessNotifyRoutineEx` can

Microsoft Confidential

Capability	Without Driver (Baseline)	With Driver (Enhanced)
		deny process creation of known-bad executables with no reboot
Tamper resistance of security service	Admin can stop or uninstall a user-mode service; admin can remove IFEO registry settings	Kernel driver is significantly harder to unload; `CmRegisterCallbackEx` can protect IFEO mitigation settings and security service configuration from tampering
File system access control	AppControl blocks execution but not file read/write	Minifilter driver can block read access to cheat tool payloads and prevent writes to protected paths
Registry protection	No protection for security-relevant registry keys	`CmRegisterCallbackEx` can monitor and block modifications to AppControl policies, security settings, and service configuration

Impact on specific attack families:

- **Co-skip (dwm.exe injection):** Without a driver, Exploit Protection CIG on dwm.exe blocks unsigned DLL injection, ACG blocks shellcode injection, and AppControl blocks the injector binary from executing. This is now a **prevention** capability at baseline — a significant improvement over detection-only. With a driver, `ObRegisterCallbacks` adds handle-level protection (preventing the injector from obtaining a write handle to dwm.exe) and `PsSetLoadImageNotifyRoutine` provides real-time alerting as an additional layer.
- **Genie Study (ctfmon.exe reflective loader):** Without a driver, Exploit Protection mitigations (Disable Extension Points, EAF, Block Low Integrity/Remote Images) harden ctfmon.exe against common injection vectors. ACG on ctfmon.exe (if compatible) blocks reflective loading. CIG is risky for ctfmon.exe due to third-party IME dependencies. With a driver, handle protection on ctfmon.exe and load-image notifications catch injection regardless of technique.
- **Furina (DLL injection into Bluebook):** CIG (Section 4a) blocks unsigned DLLs from loading into Bluebook. The kernel driver adds handle protection as a second layer — even if CIG were bypassed, the attacker cannot obtain the handle needed to inject.

Signing requirements:

A Windows kernel driver must be signed through:

- **WHQL (Windows Hardware Quality Labs):** Standard hardware driver certification through the Windows Hardware Dev Center. Requires HLK (Hardware Lab Kit) testing.

This produces a Microsoft-countersigned driver that loads on Windows 10 and 11 with Secure Boot enabled.

Recommendation: Implement the baseline (user-mode) mitigations first — they now provide meaningful **prevention** (not just detection) for system processes via Exploit Protection, with lower engineering cost than a kernel driver. Begin the kernel driver as a parallel workstream. The driver's handle protection, real-time detection, and tamper resistance for IFEO settings close the remaining gaps — particularly protecting against admin-level tampering with the Exploit Protection configuration.

Microsoft Confidential

5. Display and Screen Capture Prevention

Problem

Screen capture is the primary attack vector for AI-assisted cheating. Co-skip captures screenshots of test questions via injection into dwm.exe, transmits them to a command-and-control server, and uses AI to infer answers in real time. This is the #1 concern identified by College Board.

5a. Available Mitigations

SetWindowDisplayAffinity (P0 — Highest Priority)

The most important single mitigation available today. `SetWindowDisplayAffinity` with the `WDA_EXCLUDEFROMCAPTURE` flag instructs the Windows compositor to exclude the specified window from all standard screen capture APIs.

```
// Apply to Bluebook's main window — call immediately after window creation
SetWindowDisplayAffinity(hWnd, WDA_EXCLUDEFROMCAPTURE);
```

What it blocks:

Capture Method	Blocked?
`Windows.Graphics.Capture` API	Yes
`GetDC(NULL)` + `BitBlt` (desktop DC capture)	Yes
Magnification API	Yes
`PrintWindow`	Yes
Windows Snipping Tool / Print Screen	Yes
OBS, Discord screen share, similar tools	Yes

Availability: Windows 10 version 2004 (May 2020 Update) and later. Broad coverage on student devices.

No elevation required. The Bluebook app itself calls this on its own windows. This should be implemented immediately.

Standard user. Windows 10 2004+ and Windows 11.

AppControl Blocking of Capture Tools

During test sessions, the AppControl policy (Section 3c) should block known screen capture utilities: OBS Studio, ShareX, Greenshot, LightShot, and similar tools. While not comprehensive against custom tools, this raises the bar against casual cheating. *Requires admin/SYSTEM. Windows 10 1903+ and Windows 11.*

GetRuntimeAttestationReport for Display Driver Verification

The runtime attestation report (Section 3b) includes all loaded kernel modules, including display adapter drivers. Virtual display adapter drivers (used by remote desktop or VM configurations to create invisible virtual screens) can be detected by comparing loaded driver hashes against the allowlist. *Requires admin/SYSTEM. Windows 11 only (requires HVCI).*

Microsoft Confidential

Platform-Level Capture API Monitoring

Windows is adding platform-level signal generation for all screen capture API usage. While enforcement is not yet available, ETW tracing can detect when any process invokes capture APIs during a test session. The Bluebook Security Service can subscribe to these events and alert the Bluebook cloud service.

Relevant capture API paths being instrumented:

- `Windows.Graphics.Capture` — the modern capture API
- `GetDC(NULL)` + `BitBlt` — legacy desktop DC capture
- `Magnification API` — magnifier-based capture
- `PrintWindow` — window content capture

Requires admin/SYSTEM for ETW subscription. Windows 10 and 11.

5b. Known Gaps

The following scenarios are not fully addressed by available mitigations:

- **Kernel-level framebuffer access:** An attacker with a kernel driver can read directly from the GPU framebuffer, bypassing `SetWindowDisplayAffinity` entirely. AppControl + attestation mitigate this (by blocking/detecting the driver) but do not prevent it if the attacker has a novel signed driver.
- **DWM bypass via injection:** `SetWindowDisplayAffinity` can be bypassed by attackers who inject code into `dwm.exe` itself (the Co-skip attack vector). Because DWM is the process performing the compositor exclusion, code running within DWM's context can access content before the exclusion is applied. Preventing injection into DWM (Section 4b) is therefore critical to the effectiveness of display affinity protection.
- **Custom display drivers:** Attackers can install virtual display adapter drivers that mirror display output to a secondary (invisible) virtual screen not subject to `SetWindowDisplayAffinity`. Runtime driver attestation (Section 3b) can detect these drivers.
- **Hardware capture:** HDMI splitters, capture cards, and external cameras are out of scope for any software-based mitigation. Physical proctoring addresses this threat.

This is the area with the largest residual gap compared to Apple and ChromeOS platforms. `SetWindowDisplayAffinity` is a strong user-mode mitigation but is not equivalent to Apple's hardware-backed compositor exclusion.

---

6. Input Integrity and Remote Control Prevention

Problem

Cheat tools can inject keyboard and mouse input into the Bluebook application to automate answer entry, or attackers can use remote desktop tools to have a remote expert take the test.

Input Injection Detection

Windows marks injected input events with specific flags that applications can detect:

Microsoft Confidential

```
// In a low-level keyboard hook (WH_KEYBOARD_LL)
LRESULT CALLBACK LowLevelKeyboardProc(int nCode, WPARAM wParam, LPARAM lParam) {
    KBDLLHOOKSTRUCT* pKb = (KBDLLHOOKSTRUCT*)lParam;
    if (pKb->flags & LLKHF_INJECTED) {
        // This input was injected by SendInput or similar — flag it
    }
    if (pKb->flags & LLKHF_LOWER_IL_INJECTED) {
        // Injected from a lower integrity level process
    }
    return CallNextHookEx(NULL, nCode, wParam, lParam);
}
```

The `LLKHF_INJECTED` and `LLKHF_LOWER_IL_INJECTED` flags are set by the kernel and cannot be spoofed by user-mode code.

Standard user. Windows 10 and 11.

Low-Level Hook Detection

Enumerate active low-level hooks (`WH_KEYBOARD_LL`, `WH_MOUSE_LL`) to detect processes that are intercepting all keyboard and mouse input. While legitimate accessibility tools use hooks, unexpected hooks during a test session are suspicious. *Standard user. Windows 10 and 11.*

Remote Desktop and Remote Control Detection

Check	API / Method	Blocks
Remote desktop session	`GetSystemMetrics(SM_REMOTESESSION)`	RDP, Citrix
Known remote tools	Process enumeration for TeamViewer, AnyDesk, VNC, Chrome Remote Desktop	Third-party remote access
AppControl blocking	Policy blocking remote access tool executables	Prevents remote tools from running
Terminal Services API	`WTSEnumerateSessions` — detect non-console sessions	Detects secondary sessions

Standard user for detection APIs; admin/SYSTEM for AppControl blocking. Windows 10 and 11.

UIAccess Abuse Prevention

Programs with the `UIAccess` flag in their manifest can interact with windows of higher-integrity processes. While intended for accessibility tools, this can be exploited to read or manipulate Bluebook windows. The security service should enumerate running UIAccess processes and flag any that are not part of the standard Windows accessibility stack. *Standard user for enumeration. Windows 10 and 11.*

Clipboard Isolation

```
// At test session start
OpenClipboard(hWnd);
EmptyClipboard();
CloseClipboard();

// During test: monitor clipboard changes via WM_CLIPBOARDUPDATE
// Flag or block paste operations (WM_PASTE) during test
```

Microsoft Confidential

Clear the clipboard at test start and monitor for clipboard changes during the session. This prevents pre-loaded answer content from being pasted. *Standard user. Windows 10 and 11.*

Network Isolation

The **Windows Filtering Platform (WFP)** allows the Bluebook Security Service to install firewall rules restricting all network traffic to College Board service endpoints only during the test session.

```
// Pseudocode for WFP filter
Permit: traffic to CB attestation server IPs
Permit: traffic to CB test delivery server IPs
Permit: DNS resolution for CB domains
Block: all other outbound traffic
Block: all inbound traffic except established connections
```

This directly defeats Co-skip's C2 communication. If the cheat tool cannot reach its command-and-control server to transmit screenshots or receive AI-generated answers, the attack is neutralized even if screen capture succeeds.

WFP rules can be installed and removed dynamically by the security service — no reboot required. *Requires admin/SYSTEM. Windows 10 and 11.*

Input Injection Controls (Platform Capabilities)

The Windows platform includes input injection controls relevant to Bluebook security:

- **DoInputCheck policy layer:** Controls whether input injection is permitted, with `Feature_InputInjectionDoInputCheck` enabled by default
- **NtUserInject* API paths:** Platform-level validation on all input injection code paths
- These controls provide baseline protection; the Bluebook app's own hook-based detection (above) adds application-layer defense-in-depth

Platform-level (no app action required). Windows 10 and 11.

---

7. Peripheral, Audio, and AI Feature Controls

USB Mass Storage

Prevent use of USB storage devices to load cheat tools or exfiltrate test content during the session:

- **Device Setup Class policy:** The security service can disable the USB mass storage device setup class (`{36FC9E60-C465-11CF-8056-444553540000}`) via registry during the test session
- **AppControl:** Block execution of any code from removable media paths (`E:\`, etc.)
- **Restore after test:** Re-enable USB mass storage when the session ends

Requires admin/SYSTEM for registry and AppControl changes. Windows 10 and 11.

Audio

Prevent audio-based cheating (receiving answers via earpiece, transmitting questions via microphone):

Microsoft Confidential

- **WASAPI (Windows Audio Session API):** The security service can mute microphone input and speaker output during the test session
- **Audio session monitoring:** Enumerate active audio sessions to detect unauthorized audio communication applications
- **Restore after test:** Unmute when the session ends

Requires admin/SYSTEM for WASAPI session control. Windows 10 and 11.

Notification and System UI Suppression

Prevent notifications from leaking answers or distracting students:

- **Focus Assist / Do Not Disturb:** Enable via the security service at test session start to suppress all notifications
- **Action Center:** Disable access during the test session
- **System gestures:** Where possible, suppress edge swipe gestures (Task View, Widgets) that could expose other application content

Requires admin/SYSTEM. Windows 10 and 11 (Widgets are Windows 11 only).

---

8. Debugger and Tampering Detection

Debugger Detection

Multiple complementary checks detect whether a debugger is attached to the Bluebook process:

```
// User-mode debugger detection
if (IsDebuggerPresent()) { /* flag */ }

// Remote debugger detection (kernel debugger or cross-process)
BOOL remoteDebugger = FALSE;
CheckRemoteDebuggerPresent(GetCurrentProcess(), &remoteDebugger);
if (remoteDebugger) { /* flag */ }

// NtQueryInformationProcess — ProcessDebugPort (7)
DWORD_PTR debugPort = 0;
NtQueryInformationProcess(GetCurrentProcess(), 7,
    &debugPort, sizeof(debugPort), NULL);
if (debugPort != 0) { /* flag */ }

// NtQueryInformationProcess — ProcessDebugObjectHandle (30)
HANDLE debugObject = NULL;
NtQueryInformationProcess(GetCurrentProcess(), 30,
    &debugObject, sizeof(debugObject), NULL);
if (debugObject != NULL) { /* flag */ }
```

Standard user (self-query). Windows 10 and 11.

Microsoft Confidential

Process Signature Policy

Use `SetProcessMitigationPolicy` with `ProcessSignaturePolicy` (Section 4a) to prevent non-Microsoft-signed debuggers from attaching to the Bluebook process. *Standard user (self-hardening). Windows 10 1709+ and Windows 11.*

Runtime Integrity Verification

Bluebook can hash its own `.text` (code) sections at runtime and compare against known-good values. If code has been modified in memory (e.g., by a debugger setting breakpoints or patching code), the hashes will not match. *Standard user (self-query). Windows 10 and 11.*

Process Suspension Detection

Monitor for the process being suspended (a common debugger/analysis technique). Use a watchdog thread or timer that triggers if the main thread does not check in within an expected interval. *Standard user. Windows 10 and 11.*

---

10. Known Limitations and Residual Risk

Kernel-Level Attackers

An attacker who can load their own kernel driver can bypass all user-mode protections: read process memory directly, access the GPU framebuffer, intercept I/O, and disable mitigations. AppControl and attestation mitigate this by (a) preventing known-bad drivers from loading and (b) detecting that unauthorized drivers were loaded. The Bluebook kernel driver (Enhanced tier) further raises the bar — an attacker's driver must contend with the Bluebook driver's kernel callbacks and handle protections, creating a kernel-vs-kernel conflict that dramatically increases attack complexity. However, a novel signed driver or a zero-day in a legitimate driver remains a residual risk.

TPM Relay / Proxy Attack

A sophisticated attacker can run Bluebook inside a VM while forwarding all TPM commands to the host's physical TPM. The host TPM returns genuine quotes signed with a valid hardware AIK, and genuine PCR values reflecting the host's real boot state — including DRTM measurements if Secure Launch is enabled. From the attestation server's perspective, the evidence is indistinguishable from a bare-metal device.

This attack does not require any cryptographic forgery. The hypervisor treats TPM commands as opaque byte buffers and relays them to the host's TPM driver. Open-source hypervisors (QEMU/KVM) already have TPM passthrough infrastructure that can serve as a starting point. A skilled attacker could implement this in days to weeks.

Mitigations that help but do not eliminate the threat:

- **TLS session binding:** Include a hash of the TLS session key in the attestation nonce, binding the TPM quote to the specific connection. The attacker must relay both TLS and TPM operations consistently.

Microsoft Confidential

- **Multiple rapid challenge-response rounds:** Accumulate latency from relay round trips during the test session. Detectable but gameable with fast interconnects.
- **Layered detection signals:** Require MAA attestation + CPUID checks + SMBIOS checks + `GetRuntimeAttestationReport` + timing checks. The attacker must relay or spoof ALL channels consistently — one inconsistency exposes the attack.
- **Device identity registration:** Pre-register TPM identities (EK/AIK) to specific students. Flag if the same TPM identity appears from unexpected network contexts.

Not currently mitigated: There is no generally available mechanism on Windows client devices to cryptographically prove that the requesting software and the TPM are on the same physical machine. Future confidential compute attestation (AMD SEV-SNP, Intel TDX) may close this gap, but these technologies are not available on student laptops today.

Hardware Capture

HDMI splitters, capture cards, and external cameras are out of scope for software-based mitigations. Physical proctoring or environmental controls address this threat.

Local Administrator Privileges

Students with local admin can potentially disable mitigations: uninstall the security service, remove AppControl policies, or disable security features. The kernel driver (Enhanced tier) mitigates this by protecting the security service from termination and monitoring registry changes to security policies. Attestation provides an additional layer — if expected mitigations are not active at test time, the session is flagged. Detection is as valuable as prevention when test scores can be invalidated.

AppControl Reboot Friction

Kernel-mode AppControl enforcement requires a reboot to activate and deactivate. This introduces friction in the test session workflow. User-mode-only CI policies (where applicable) avoid this, but provide less comprehensive protection.

Defense-in-Depth Value Proposition

While no single layer is impenetrable against a determined admin-level attacker, the combination of mitigations in this document dramatically raises the cost, complexity, and skill required to cheat. The cheating industry operates on economic incentives — making Windows cheating significantly harder shifts the cost-benefit calculation and reduces the attack surface from a commoditized operation to one requiring advanced exploitation skills.

Microsoft Confidential